Chapter 4.8: Control Hazards

Branch Prediction and Flushing

Contents

1 Introduction

> *"To make a computer with automatic program-interruption facilities behave [sequentially] was not an easy matter, because the number of instructions in various stages of processing when an interrupt signal occurs may be large."*
> — Fred Brooks, Jr., *Planning a Computer System: Project Stretch*, 1962

Thus far, we have limited our concern to hazards involving arithmetic operations and data transfers. However, there are also pipeline hazards involving **conditional branches**.

Control Hazard (Branch Hazard)

A **control hazard** is a delay in determining the proper instruction to fetch, caused by a conditional branch instruction. An instruction must be fetched at every clock cycle to sustain the pipeline, yet in our basic design the decision about whether to branch doesn't occur until the **MEM pipeline stage**.

1.1 Why Control Hazards are Different

This section on control hazards is shorter than the previous sections on data hazards for three reasons:

1. Control hazards are **relatively simple** to understand.
2. They occur **less frequently** than data hazards.
3. There is **nothing as effective** against control hazards as forwarding is against data hazards.

Hence, we use **simpler schemes**. We look at two schemes for resolving control hazards and one optimization.

2 The Problem: Branch Delay

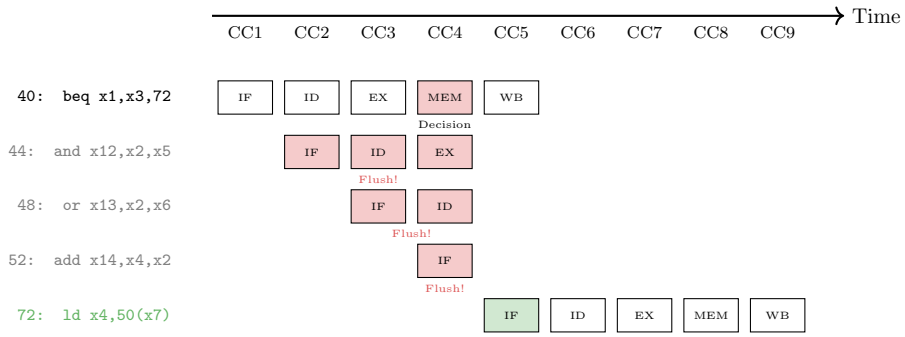

Figure 1: **Figure 4.59:** The impact of the pipeline on a branch instruction. Without optimization, the branch decision is made in the MEM stage (CC4), so **three sequential instructions** are fetched and must be flushed if the branch is taken.

The Problem

Without intervention, three following instructions will begin execution **before** the `beq` instruction determines whether to branch to location 72.

3 Scheme 1: Assume Branch Not Taken

Stalling until the branch is complete is too slow. A better improvement is to **predict that the conditional branch will not be taken** and continue execution down the sequential instruction stream.

- If the branch is **not taken**: Prediction correct, no penalty.
- If the branch is **taken**: The instructions that were fetched and decoded must be **discarded** (flushed), and execution continues at the branch target.

3.1 Flushing Instructions

Flush

To **flush** instructions means to discard them from the pipeline, usually due to an unexpected event like a mispredicted branch.

To discard instructions, we change the original control values to **0s**, similar to what we did for load-use data hazards. The difference is that we must change the instructions in **IF, ID, and EX stages** when the branch reaches the MEM stage.

4 Reducing the Delay of Branches

One way to improve conditional branch performance is to reduce the cost of a taken branch. If we move the conditional branch execution **earlier in the pipeline**, fewer instructions need to be flushed.

4.1 Moving Branch Decision to ID Stage

Moving the branch decision up requires two actions:

1. **Computing the branch target address**: Move the branch adder from EX stage to ID stage. We already have the PC and immediate field in the IF/ID pipeline register.
2. **Evaluating the branch decision**: For `beq`, compare two register reads during ID using an **equality test unit** (XOR bits, then OR the results).

4.2 Complications

Moving the branch test to ID introduces new hardware requirements:

New Forwarding and Hazard Detection

1. **Forwarding to ID**: The equality test unit may need results from EX/MEM or MEM/WB pipeline registers. New forwarding logic is required.
2. **New Stall Conditions**:
   - ALU instruction immediately before branch → **1-cycle stall**
   - Load instruction immediately before branch → **2-cycle stall**

4.3 The IF.Flush Control Line

To flush instructions in the IF stage, we add a control line called `IF.Flush` that **zeros the instruction field** of the IF/ID pipeline register. This transforms the fetched instruction into a **nop** (no operation).

Example: Pipelined Branch (Optimized)

Consider this instruction sequence with branch execution moved to ID stage:

```
36  sub  x10, x4, x8
40  beq  x1, x3, 16    // Branch to 40+16*2=72
44  and  x12, x2, x5
48  or   x13, x2, x6
...
72  ld   x4, 50(x7)    // Branch target
```

Result: If branch is taken, only **one instruction** (the one being fetched at CC3) needs to be flushed, instead of three.

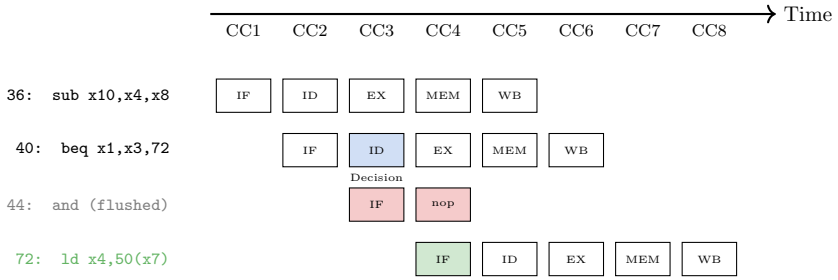

Figure 2: **Figure 4.60:** With branch execution moved to ID stage, only **one bubble** is needed on a taken branch instead of three.

5 Dynamic Branch Prediction

Assuming a conditional branch is not taken is a simple form of **static** branch prediction. With deeper pipelines and multiple issue, a simple static prediction scheme wastes too much performance. With more hardware, it is possible to **predict branch behavior during program execution**.

Dynamic Branch Prediction

Prediction of branches at runtime using runtime information. The approach looks up the address of the instruction to see if the branch was taken the last time it was executed.

5.1 Branch Prediction Buffer (Branch History Table)

Branch Prediction Buffer

A **branch prediction buffer** (or **branch history table**) is a small memory indexed by the lower portion of the branch instruction's address. It contains one or more bits indicating whether the branch was recently taken or not.

Note: The prediction is just a **hint**. We don't even know if the bit was set by this branch or another branch with the same low-order address bits. If the hint is wrong:

1. Incorrectly predicted instructions are deleted.
2. The prediction bit is **inverted** and stored back.
3. The proper sequence is fetched and executed.

5.2 The Problem with 1-Bit Prediction

1-Bit Prediction Shortcoming

Even if a branch is almost always taken (like a loop), a 1-bit scheme can mispredict **twice** per loop execution rather than once:

- On the **first iteration**: The bit was flipped on the previous loop exit.
- On the **last iteration**: The branch is not taken, but the bit predicts taken.

Example: Loop Prediction Accuracy

Consider a loop branch that branches 9 times in a row, then is not taken once.
With 1-bit prediction:

- Mispredicts iteration 1 (bit was flipped on last exit)
- Mispredicts iteration 10 (exit iteration)
- Accuracy: $\frac{8}{10} = 80\%$

Ideally: Accuracy should match the 90% taken frequency.

5.3 2-Bit Branch Prediction

To remedy the 1-bit weakness, **2-bit prediction schemes** are often used. A prediction must be wrong **twice** before it is changed.

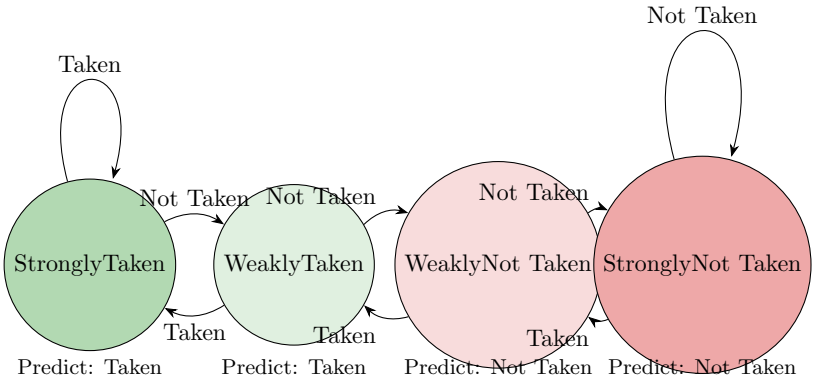

Figure 3: **Figure 4.61:** The states in a 2-bit prediction scheme. A prediction must be wrong twice before it changes. This is a **counter-based predictor**: incremented on accurate prediction, decremented otherwise.

Result: A branch that strongly favors taken or not taken will be mispredicted **only once** when its pattern is briefly interrupted.

6 Advanced Prediction Techniques

6.1 Branch Target Buffer

A branch predictor tells us *whether* a branch is taken, but still requires calculation of the branch target. A **branch target buffer** caches the destination PC or instruction.

Branch Target Buffer

A structure that caches the destination PC or destination instruction for a branch. It is usually organized as a **cache with tags**, making it more costly than a simple prediction buffer.

6.2 Correlating Predictor

The 2-bit scheme uses only local information about a particular branch. A **correlating predictor** combines:

- **Local behavior**: How this specific branch has behaved.
- **Global information**: How recently executed branches have behaved.

6.3 Tournament Branch Predictor

A **tournament predictor** uses multiple prediction mechanisms and a selector that chooses which predictor to use for each branch based on which has been more accurate.

7 Reducing Branches: Conditional Move

One way to reduce the number of conditional branches is to add **conditional move instructions**. Instead of changing the PC with a conditional branch, the instruction conditionally changes the destination register.

ARMv8 CSEL Instruction

```
CSEL X8, X11, X4, NE
```

- If condition NE (not equal) is true: `X8 = X11`
- If condition is false: `X8 = X4`

This avoids a branch entirely!

8 Check Yourself

Consider three branch prediction schemes with a 2-cycle misprediction penalty and 0-cycle correct prediction penalty:

- **Predict Not Taken**
- **Predict Taken**
- **Dynamic Predictor** (90% accurate)

Taken %	Predict NT	Predict T	Dynamic 90%	Best Choice
5%	95% correct	5% correct	90% correct	**Predict Not Taken**
95%	5% correct	95% correct	90% correct	**Predict Taken**
70%	30% correct	70% correct	90% correct	**Dynamic**

Table 1: Choosing the best branch prediction scheme based on taken frequency.

9 Flashcards

1. **Q: What is a control hazard in a processor pipeline?**
   A: It is a delay in determining the proper instruction to fetch, caused by a conditional branch instruction.

2. **Q: In the non-optimized five-stage pipeline described, in which stage is the decision about whether to branch made?**
   A: The decision is made in the MEM (Memory Access) pipeline stage.

3. **Q: Without any optimization, how many sequential instructions following a branch are fetched before the branch decision is made in the MEM stage?**
   A: Three sequential instructions are fetched and begin execution.

4. **Q: The technique of discarding instructions in a pipeline, usually due to an unexpected event like a mispredicted branch, is called _____.**
   A: flushing

5. **Q: What is the simplest static branch prediction scheme mentioned in the text?**
   A: The 'Assume Branch Not Taken' scheme, where execution continues down the sequential instruction stream.

6. **Q: In the 'Assume Branch Not Taken' scheme, what happens if the conditional branch is actually taken?**
   A: The instructions that were being fetched and decoded must be discarded, and execution continues at the branch target.

7. **Q: How are instructions discarded or flushed when a branch is mispredicted in the 'Assume Not Taken' scheme?**
   A: The original control values for the instructions in the IF, ID, and EX stages are changed to 0s.

8. **Q: What is the primary method discussed to improve conditional branch performance by reducing the cost of a taken branch?**
   A: Moving the conditional branch execution earlier in the pipeline, from the MEM stage to the ID stage.

9. **Q: What are the two actions that must occur earlier to move conditional branch execution to the ID stage?**
   A: Computing the branch target address and evaluating the branch decision must both occur earlier.

10. **Q: What hardware modification is needed in the ID stage to calculate the branch target address earlier?**
    A: The branch adder is moved from the EX stage to the ID stage.

11. **Q: What hardware modification is needed in the ID stage to evaluate the branch decision for 'branch if equal' earlier?**
    A: An equality test unit is added to compare two register values during the ID stage.

12. **Q: When branch execution is moved to the ID stage, what new hardware complication arises regarding data dependencies?**
    A: New forwarding logic is required, as the equality test unit in ID may need results from later pipeline stages (EX/MEM or MEM/WB).

13. **Q: If an ALU instruction immediately precedes a branch that depends on its result, what is required when branch execution is in the ID stage?**
    A: A stall will be required because the ALU result is produced in the EX stage, after the branch's ID cycle.

14. **Q: If a load instruction is immediately followed by a conditional branch that depends on its result, how many stall cycles are needed if branch execution is in the ID stage?**
    A: Two stall cycles are needed, as the load result is not available until the end of the MEM cycle.

15. **Q: By moving branch execution to the ID stage, the penalty of a taken branch is reduced to how many instructions?**
    A: The penalty is reduced to only one instruction, which is the one currently being fetched.

16. **Q: What is the purpose of the 'IF.Flush' control line?**
    A: It zeros the instruction field of the IF/ID pipeline register, effectively transforming the fetched instruction into a nop.

17. **Q: Prediction of branches at runtime using runtime information is known as _____.**
A: dynamic branch prediction

18. **Q: What is a branch prediction buffer, also known as a branch history table?**
A: It is a small memory indexed by the lower portion of a branch instruction's address, containing bits that indicate if the branch was recently taken.

19. **Q: What is the primary performance shortcoming of a simple 1-bit dynamic branch prediction scheme?**
A: If a branch is almost always taken (like in a loop), it will be mispredicted twice for each full loop execution rather than just once.

20. **Q: For a loop branch that is taken nine times and not taken once, what is the prediction accuracy using a 1-bit scheme?**
A: The accuracy is 80%, with two incorrect predictions (on the first and last iterations) and eight correct ones.

21. **Q: Why does a 1-bit predictor mispredict the first iteration of a loop that was just completed?**
A: The prediction bit was flipped to 'not taken' on the final, exiting iteration of the previous loop execution.

22. **Q: How does a 2-bit branch prediction scheme improve over a 1-bit scheme?**
A: A prediction must be wrong twice before the prediction state is changed, leading to only one misprediction for highly regular loops.

23. **Q: In a 2-bit counter-based predictor, what action is taken when the prediction is accurate?**
A: The counter is incremented (or moved towards a more strongly-taken/not-taken state).

24. **Q: In a 2-bit counter-based predictor, what action is taken when the prediction is inaccurate?**
A: The counter is decremented (or moved towards the weakly-taken/not-taken state, and eventually flips).

25. **Q: What structure caches the destination PC or destination instruction for a branch to reduce the taken-branch penalty?**
A: A branch target buffer.

26. **Q: How does a branch target buffer differ from a simpler branch prediction buffer?**
A: It is usually organized as a cache with tags, making it more costly, and it stores the target address or instruction, not just a prediction bit.

27. **Q: What is a correlating predictor?**
A: A branch predictor that combines the local behavior of a branch with global information about recently executed branches.

28. **Q: What is a tournament branch predictor?**
A: A predictor that uses multiple prediction schemes and a selection mechanism to choose the best one for a given branch.

29. **Q: What type of instruction can be added to an instruction set to reduce the number of conditional branches?**
A: Conditional move instructions.

30. **Q: What does the ARMv8 'CSEL' instruction do?**
A: It conditionally selects one of two source registers to copy to a destination register based on a condition code.

31. **Q: For a branch with 5% taken frequency and a 2-cycle misprediction penalty, which scheme is best: predict-taken, predict-not-taken, or 90% accurate dynamic?**
A: The predict-not-taken scheme is best, as it will be correct 95% of the time.

32. **Q: For a branch with 95% taken frequency and a 2-cycle misprediction penalty, which scheme is best: predict-taken, predict-not-taken, or 90% accurate dynamic?**
A: The predict-taken scheme is best, as it will be correct 95% of the time.

33. **Q: For a branch with 70% taken frequency and a 2-cycle misprediction penalty, which scheme is best: predict-taken, predict-not-taken, or 90% accurate dynamic?**
A: The dynamic predictor is best, as its 90% accuracy is higher than predict-taken (70%) or predict-not-taken (30%).

34. **Q: What is the name for a pipeline hazard involving conditional branches?**
A: It is called a control hazard or a branch hazard.

35. **Q: The delay in determining the proper instruction to fetch due to a branch instruction is called a _____ or branch hazard.**
A: control hazard

36. **Q: Why is the section on control hazards presented as shorter than the one on data hazards?**
A: Control hazards are simpler to understand, occur less frequently, and have less effective resolution schemes compared to forwarding for data hazards.

37. **Q: What is a simple static branch prediction scheme that improves upon stalling?**
A: The 'assume branch not taken' scheme, where execution continues down the sequential instruction stream.

38. **Q: What action is taken to discard instructions in a pipeline when a branch is mispredicted?**
A: The original control values for the incorrect instructions are changed to 0s.

39. **Q: Term: Flush**
A: Definition: To discard instructions in a pipeline, usually due to an unexpected event like a mispredicted branch.

40. **Q: In the unoptimized pipeline, which stages must be flushed when a branch taken in the MEM stage is discovered?**
A: The instructions in the IF, ID, and EX stages must be flushed.

41. **Q: What is one way to improve conditional branch performance by reducing the cost of a taken branch?**
A: Move the conditional branch execution earlier in the pipeline, such as to the ID stage.

42. **Q: How is the branch target address calculation moved to the ID stage?**
A: The branch adder is moved from the EX stage to the ID stage, using the PC and immediate field from the IF/ID register.

43. **Q: What is the more difficult part of moving branch execution to the ID stage?**
A: Moving the branch decision itself, which involves comparing two register values during the ID stage.

44. **Q: Moving the branch test to the ID stage introduces the need for new _____ and _____ hardware.**
A: forwarding, hazard detection

45. **Q: If branch comparison is moved to ID, from which pipeline registers can the bypassed source operands come?**
A: The bypassed operands can come from either the EX/MEM or MEM/WB pipeline registers.

46. **Q: If branch execution is moved to the ID stage, what happens if an ALU instruction immediately preceding a branch produces an operand for the branch test?**
A: A data hazard occurs, and a one-cycle stall is required.

47. **Q: If branch execution is moved to the ID stage, what is the penalty for a load instruction immediately followed by a branch that depends on the load result?**
A: A two-cycle stall is needed.

48. **Q: What is the primary benefit of moving conditional branch execution to the ID stage?**
A: It reduces the penalty of a taken branch to only one instruction.

49. **Q: What is the purpose of the IF.Flush control line?**
A: It zeros the instruction field of the IF/ID pipeline register to flush the instruction currently in the IF stage.

50. **Q: Clearing the IF/ID register with IF.Flush transforms the fetched instruction into a _____, an instruction that has no action.**
A: nop

51. **Q: What is the term for predicting branches at runtime using runtime information?**
A: Dynamic branch prediction.

52. **Q: Is a prediction from a simple branch prediction buffer guaranteed to be for the correct branch instruction?**
A: No, another branch with the same low-order address bits may have set the prediction bit, but this does not affect correctness.

53. **Q: What happens in a 1-bit dynamic prediction scheme when a prediction turns out to be wrong?**
A: The incorrectly predicted instructions are deleted, the prediction bit is inverted, and the proper sequence is fetched.

54. **Q: What is the performance shortcoming of a simple 1-bit prediction scheme, especially with loops?**
A: It can mispredict twice for a frequently-taken loop: once on the first iteration and once on the final, non-taken iteration.

55. **Q: For a loop branch that is taken 9 times and not taken once, what is the prediction accuracy of a 1-bit scheme?**
A: The accuracy is 80%, with two incorrect predictions (first and last iteration) and eight correct ones.

56. **Q: How does a 2-bit branch prediction scheme improve upon a 1-bit scheme?**
A: A prediction must be wrong twice before it is changed, making it more resilient to single changes in branch behavior.

57. **Q: In a 2-bit prediction scheme, a branch that strongly favors taken or not taken will be mispredicted how many times when its pattern is briefly interrupted?**
A: It will be mispredicted only once.

58. **Q: A 2-bit scheme is a general instance of a _____ predictor, which is incremented on an accurate prediction and decremented otherwise.**
A: counter-based

59. **Q: Term: Branch Target Buffer**
A: Definition: A structure that caches the destination PC or destination instruction for a branch to reduce or eliminate the taken-branch penalty.

60. **Q: What kind of predictor combines the local behavior of a branch with global information about recently executed branches?**
A: A correlating predictor.

61. **Q: Term: Correlating Predictor**
A: Definition: A branch predictor that combines local behavior of a particular branch and global information about the behavior of some recent number of executed branches.

62. **Q: Term: Tournament Branch Predictor**
A: Definition: A predictor with multiple prediction mechanisms for each branch and a selector that chooses which one to use based on which has been more accurate.

63. **Q: What is an architectural alternative to conditional branches that can reduce their frequency?**
A: Conditional move instructions, which conditionally change a destination register's value instead of the PC.

64. **Q: What is the function of the ARMv8 CSEL instruction?**
A: It conditionally selects one of two source registers to copy to a destination register based on the condition codes.

65. **Q: Given a 2-cycle misprediction penalty and 0-cycle correct prediction penalty, which scheme is best for a branch taken with 5% frequency?**
A: Predict not taken is best, as it is correct 95% of the time, yielding a lower average penalty than the 90% accurate dynamic predictor.

66. **Q: Given a 2-cycle misprediction penalty and 0-cycle correct prediction penalty, which scheme is best for a branch taken with 95% frequency?**
A: Predict taken is best, as it is correct 95% of the time, yielding a lower average penalty than the 90% accurate dynamic predictor.

67. **Q: Given a 2-cycle misprediction penalty and 0-cycle correct prediction penalty, which scheme is best for a branch taken with 70% frequency?**
A: Dynamic prediction is best, as its 90% accuracy is better than both predict taken (70%) and predict not taken (30%).